CS 6210 Fall 2025 Test 1

(120 min Canvas Quiz)

[Total Points: 106]

Table of Contents

OS Structure [21 points] 1

Exokernel [4 points] 2

Spin [6 points] 2

L3 Microkernel [11 points] 3

Virtualization [29 points] 5

Paravirtualization [15 points] 5

Full Virtualization [6 points] 7

Memory Management for Virtual Machines [8 points] 8

Parallel Systems [35 points] 9

Shared Memory [4 points] 9

Lock/Barriers [13 points] 10

LRPC and Scheduling [12 points] 12

Tornado [6 points] 14

HIDDEN - Potpourri [21 points] 15

Full Virtualization [2 points] 15

Shared Memory [3 points] 15

Locks/Barriers [11 points] 15

Cellular Disco [5 points] 18

OS Structure [21 points]

Exokernel [4 points]

1. [4 points] A library OS "B", on top of Exokernel experiences memory pressure and needs additional physical memory. Exokernel decides to revoke 20 physical page frames from library OS "A" to service this request.

Describe how Exokernel can revoke the physical page frames from library OS "A" and how library OS "A" will respond.

+2 if the following is conveyed:

- Exokernel can make a revoke upcall (or repossession exception) to library OS "A" with a "repossession vector" containing the 20 physical page frames it is reclaiming from library OS "A".

+2 if the following is conveyed:

- Library OS "A" will use the "repossession vector" to take corrective action and update any mappings using these physical page frames. For example, the library OS "A" can write the contents of 20 physical page frames to disk.

Spin [6 points]

1. [6 points] Your friend -- having not read a research paper in 3 years -- is struggling to understand the specific benefits SPIN provided in the "Extensibility, Safety and Performance in the SPIN Operating System" paper that was assigned. He was hoping you could briefly describe (in a sentence or two) each attribute listed in title of the paper, since the paper's title was the only part that really stuck with him.

[2 points] How is SPIN extensible?

+2 if any of the following are conveyed:

- Events and event handlers: Ability of extensions to associate handlers with the events which are software interrupts (or upcalls) from SPIN into the extensions
- Modularity exposes only well-defined interfaces and hide details of extension implementations which can be loaded up on the fly

[2 points] How is SPIN safe?

+2 if any of the following are conveyed:

- Using a strongly typed language (in this case Modula-3) that provides compile time checks and runtime verification for the safety of the extensions.

- Extensions bound in Modula-3 have the necessary guarantees of memory isolation from other subsystems, yet the resolve mechanism in SPIN dynamically stitches these extensions together to allow performant cross-domain communication.

[2 points] How is SPIN performant?

There are varied aspects of performance listed and described in the paper -- +2 for any of the following:

- Extensions are dynamically linked into SPIN's hardware address space ensuring performance.
- For virtual memory management, SPIN allows extensions to define application specific system calls
- As an example, an application page-fault is caught by the hardware and passed up through SPIN's event handlers to the subsystems of the library OS that deal with page fault handling all of which are in the same hardware address space thus not requiring any "border crossing."
- As another example, processing network packets happens entirely within the extended SPIN kernel instead of user space, where it has low-latency access to both the device and data

L3 Microkernel [11 points]

1. [3 points] Greg, a core Linux contributor, is skeptical about Microkernel. He mentions that the 100x cost of the Protected Procedure Calls between different servers due to Address-Space Switching are too high to be able to achieve good performance. Provide two points as counterarguments to Greg's point of view.

Ans:(Students can provide any two valid answers for full credit)

- [+1] Mention Tagged TLBs have zero costs for address space switches.
- [+1] If Tagged TLB aren't available, exploit hardware features like segment registers
- [+1] Shared communication regions across protection domains ala LRPC
- [+1] If the calls are across LARGE protection domains the explicit cost of address space switching (including flushing TLB if necessary) may be less detrimental to performance when compared to the implicit cost (loss of locality leading to high cache misses which would happen even in a monolithic kernel)
- [+1] For any other valid reason

2. [2 points] Mention two advantages of Microkernel Approach over Monolith Kernels.

Ans: (Students can provide any two valid answers for full credit)

- [+1] Enforces a clear interface leading to a more modular system structure
- [+1] Better Process Isolation. (i.e. better process isolation leads to better security)
- [+1] More flexibility/portability
- [+1] For any other valid reason, like formal verification, better security, maintainability.
- [+1] Smaller attack surface leading to less security vulnerability since most of the core functionalities are above the microkernel
- [+1] Extensibility since individual subsystems (e.g., memory management) can be tailored to application needs.

3. [6 points] You are building an OS using a microkernel-based approach following the principles of the L3 microkernel. The processor architecture you are building this OS for has the following features:

- A 32-bit hardware address space.
- Paged virtual memory system (8KB pages) with a processor register called PTBR that points to the page table in memory to enable hardware address translation.
- A TLB that doesn't support tagging entries with address space IDs
- A pair of hardware-enforced segment registers (lower and upper bound of virtual addresses) which limit the virtual address space that can be accessed by a process running on the processor.
- A virtually indexed physically tagged processor cache.

You end up with the following subsystems that each need to be a separate protection domain.

- A: 2^30 bytes
- B: 2^30 bytes
- C: 100 * 2^20 bytes
- D: 500 * 2^20 bytes
- E: 1000 * 2^20 bytes
- F: 2000 * 2^20 bytes

These subsystems are packed into two hardware address spaces: protection domains A and B in the first; and protection domains C, D, E, and F in the second. Based on this grouping, answer the following questions.

a) [2 points] There is a context switch from A to B. What does your OS do to facilitate this context switch?

Ans:

- [+2] Load the lower and upper bounds into hardware-enforced segment registers with the lower and upper addresses of C.
- [-1] Any mention of flushing of TLB or cache or both mentioned
- [-1] Any mention of updates to PTBR

b) [2 points] There is a context switch from A to C. What does your OS do to facilitate this context switch?

Ans:

- [+1] Set PTBR to point to address space of C and flushing of TLB
- [+1] Load lower and upper bound segment register from the PCB of C
- [-1] No mention of TLB Flushing or any mention of cache flush

c) [2 points] What is the most expensive context switch in your design? Why?

Ans:

- [+1] Mention of switching between (A, B) hardware address space and (C, D, E, F) hardware address space
- [+1] Mention of loss of locality as the domain warms up.

Virtualization [29 points]

Paravirtualization [15 points]

1. [3 points]

In a para virtualized setting, the Guest OS is aware of the hypervisor and knows that the physical memory given to it may not be contiguous. Given this fact, why are shadow page tables used even in para virtualized settings?

Ans:

- We want to minimize the number of code changes to the guest OS such that it can be run on the hypervisor.
- Several places in the OS might assume contiguous physical memory.
- Hence, despite the para-virtualized setting, a shadow page table is maintained (usually by the guest OS itself) to reduce the code changes and make the guest OS easy to maintain and evolve.

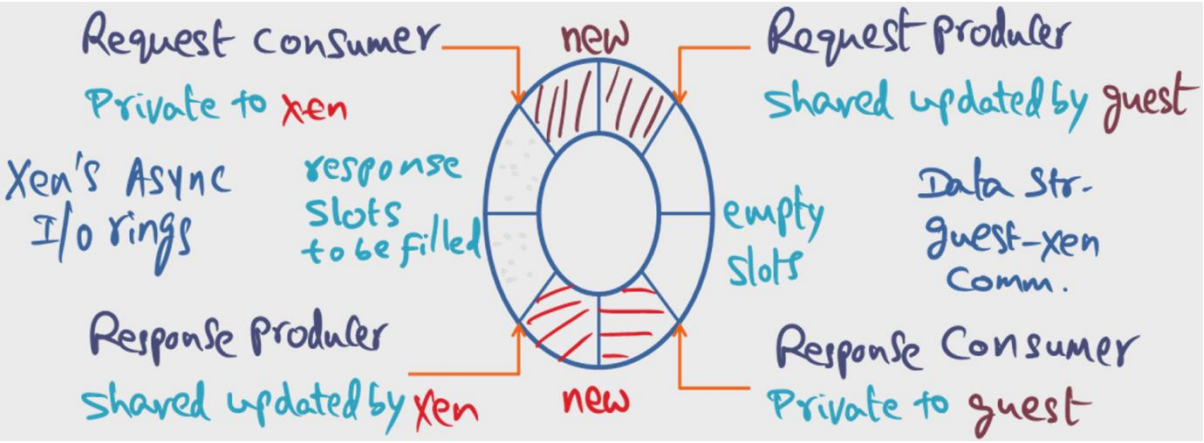
2. [5 points]

You are launching an application inside a para virtualized VM. Your expectation is that your application will have memory isolation and protection from other applications on the same VM. With succinct bullets, explain how this expectation is fulfilled.

Ans:

- The guest OS allocates a page frame as the page table data structure for the newly created process
- It initializes the page table (for simplicity assume it makes all the page table entries as "invalid")
- It uses an API provided by the hypervisor (e.g., "Create PT") to give the page frame address (which contains the page table for the newly created process) to the hypervisor.
- The hypervisor records this address as one of the valid page table pointers for this guest OS
- Whenever the guest OS wishes to schedule this process, it will use an API provided by the hypervisor (e.g., "Switch PT") to tell the hypervisor to load the PTBR with the address of the page table it has registered with the hypervisor earlier.
- When a process has a page fault, the hypervisor upcalls the guest OS which resolves the page fault and presents a valid mapping (VPN to MPN) to the hypervisor using an API (e.g., "Install Mapping") which is entered into the page table named by the API call.
- Since each process has its own page table and thus its own address space, it has guarantees of isolation and protection.

3. [7 points]



Above picture shows the I/O ring data structure used in Xen to facilitate communication between the guest OS and Xen. Guest-OS places a request in the I/O ring using the "Request Producer" pointer. Xen places a response in the I/O ring using the "Response Producer" pointer.

a) [1 point] A guest OS is polling the ring. What exact condition tells it that a response is available to consume, and what simple step should it take before reading the descriptor?

Response available when Response Producer ≠ Response Consumer (the producer has advanced beyond what the guest has consumed). The guest then reads the response entry (after ensuring the usual producer/consumer ordering so it sees the filled descriptor).

b) [2 points] Why does Xen typically not run out of space to place a response on this ring? State the invariant briefly.

Xen reuses the processed request's slot to write its response; each handled request frees exactly one slot for its response, so Xen doesn't need extra ring capacity beyond the bounded buffer.

c) [2 points] How are notifications delivered for new requests/responses, and how can the guest reduce interrupt overhead?

Xen uses event channels (asynchronous upcalls) to notify the other side that producer indices advanced; the guest can batch updates and mask/enable the channel to coalesce interrupts (or poll whenever it wants).

d) [2 points] You are implementing disk I/O in a guest OS using Xen's I/O ring. What steps are needed to achieve zero-copy semantics for a block write?

- Guest places a reference to its memory page in the I/O ring descriptor (rather than copying data).
- Guest OS pins that page and programs the device DMA engine to write directly from it.

Full Virtualization [6 points]

1. [2 points] "Physical memory is an illusion of the quest OS in a fully virtualized environment". Explain what this means.

Ans:
- The hypervisor is in control of the hardware memory which is called "machine memory".

- The guest OS's physical memory is mapped to the machine memory by the hypervisor.

- That’s the reason physical memory is an illusion of the guest of since it “thinks” it is running on real hardware.

2. [4 points] An engineering firm is running a critical, unmodified legacy CAD application inside a fully virtualized VM on an older x86 server. The application frequently makes privileged calls that interact directly with the hardware. An administrator notices poor performance and learns that this specific, older x86 architecture has a "quirk" where certain privileged instructions, when executed by the guest OS (which is running in an unprivileged mode), fail silently instead of causing a trap to the hypervisor.

a) [2 points] Explain why the standard trap-and-emulate strategy for CPU virtualization would not work correctly in this scenario?

Ans: [+1] Trap and Emulate relies on the CPU hardware to automatically trap (i.e., transfer control) to the hypervisor whenever the guest OS attempts to execute a privileged instruction.

[+1] because the privileged instructions on the older x86 architecture fail silently instead of trapping, the hypervisor is never notified.

b) [2 points] What technique does VMware ESX use to overcome this architectural limitation and ensure the guest OS functions correctly?

Ans: [+1] binary rewriting. Before the guest OS code is executed, the hypervisor scans it for these problematic privileged instructions.

[+1] It then rewrites those instructions, replacing them with code that will explicitly cause a trap or call into the hypervisor.

Memory Management for Virtual Machines [8 points]

1. [3 points] A datacenter uses ballooning to allocate or reclaim memory from VMs. The hypervisor has a policy that taxes 20% of a VM’s idle memory, starting with the VM that has the most idle memory. The hypervisor can tax over multiple rounds until the request of a VM is met.

Consider this scenario:

- VM1 requests 100 MB of additional memory.
- The hypervisor has no free machine memory
- VM2 has an idle memory of 140 MB

- VM3 has an idle memory of 200 MB

List the steps taken by the hypervisor to satisfy VM1's request.

Ans:

a) [+1] Inflate balloon driver on VM3 to get 40 MB (20% of 200 MB)

b) [+1] Inflate balloon driver on VM3 to get 32 MB (20% of 160 MB)

c) [+1] Inflate balloon driver on VM2 to get 28 MB (20% of 140 MB)

2. [2 points] Mention one advantage and one disadvantage of having a high tax rate for reclaiming unused memory from the VMs.

Ans:

[+1] Advantage: The hypervisor can quickly free up large amounts of memory and speed up urgent memory requests.

[+1] Disadvantage: A VM which was the victim of such a high taxation rate would suffer from significant performance loss if its client processes suddenly have a rapid increase in their working sets.

3. [3 points] Does VM-oblivious page sharing violate the memory isolation guarantees of the VMs? Justify your answer.

Ans:

(+1) No.

(+2) Upon a successful match the VPNs in the respective VMs will point to the same MPN. However, the entries will be marked "copy on write". If any of the VMs try to modify the page, then it will result in an access violation. The hypervisor will ensure that the pages are no longer shared across the VMs, thus guaranteeing memory isolation for the VMs.

Parallel Systems [35 points]

Shared Memory [4 points]

1. [4 points] All the variables in the execution shown below are in the shared memory of a cache coherent NUMA multiprocessor. The multiprocessor implements sequential consistency. All variables are initialized to 0 before execution starts.

Execution on processor P1	Execution on processor P2
x = 20	w = y + x
y = 30	z = x+10

Which of the following final values are **impossible** with the above execution? (+2 for correct choice; -1 for incorrect choice)

```
(i)   w = 0;  z = 10;
(ii)  w = 0;  z = 30;
(iii) w = 20; z = 30;
(iv)  w = 30; z = 10;
(v)   w = 30; z = 30;
(vi)  w = 50; z = 30;
```

Ans:

- iv and v are the correct choices

Lock/Barriers [13 points]

1. [2 point] Consider a cache-coherent multiprocessor. Assume the multiprocessor offers an atomic decrement operation on a shared variable. You decide to provide a barrier synchronization primitive in your OS that is based on the following count-based algorithm:

```
int count = N; // initialized to the number of participating threads
barrier()
0.{
1.    decrement(count);
2.    if (count == 0)
3.        count = N;                // last processor to arrive
4.    else
5.       while (count > 0);        // wait for last process
6.      while (count!= N);//to not leave barrier before count reset
7.}
```

Identify any two race conditions that will lead to the algorithm not working as written above.

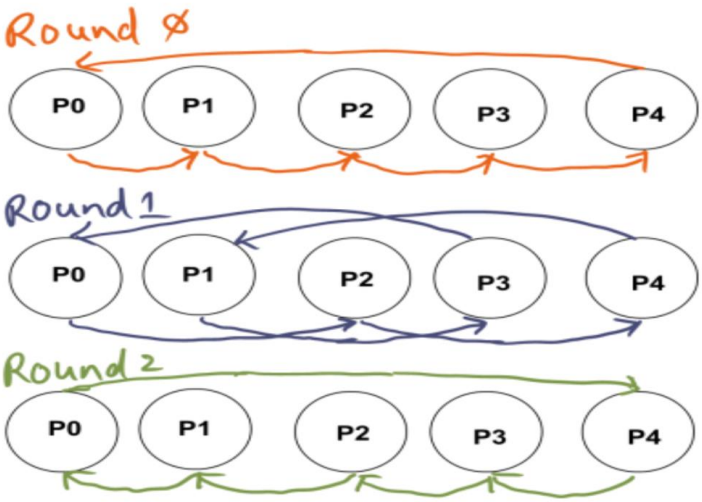
Ans:

(+1) The last processor may reset count to N (line 3) before the early arriving processors have seen that the count became 0 (line 5).

(+1) The last processor may race to the next barrier and decrement count (line 1) before some of the early arriving processors have noticed that the count has been reset to N (line 7).

2. [6 points]

a. [2 points] Given the following picture of the dissemination algorithm, why is the barrier not completed after Round 0?



Ans:

Each processor advances independently to the next round as soon as it has arrived at the barrier, and it has received an "arrived" message from the processor to its immediate left.

b. [2 points] Focusing on node P2, what information does P2 know at the end of Round 1? Explain your answer.

Ans:

- [+0.5] At the end of Round 0, P2 knows (P1 and P2) have arrived.
- [+0.5] In Round 1, P2 learns P0 has arrived at the barrier.
- [+0.5] At the end of Round 0, P0 knows that (P4, P0) have arrived at the barrier
- [+0.5] Thus, at the end of round 1, P2 knows that {P0, P1, P2, P4} have arrived at the barrier.

c. [1 point] What is the minimum number of additional processors that would warrant a Round 3?

Ans: [+1] 4 additional processors

d. [1 point] How many fewer processors should be participating to complete the barrier at the end of Round 1

Ans: [+1] 1 less processor.

3. [2 points] Name two important properties that an implementation of the mutual exclusion lock algorithm should satisfy.

Ans: Multiple answers can be accepted here

- the lock can be gotten quickly when no one else is using it

- during high contention for a lock, handoff from the current holder to the next should happen quickly with the least amount of disruption

- only one holder at any given time

4. [3 points] Consider Anderson's array-based queuing lock shown pictorially below, where N is the number of processors.



One disadvantage of this algorithm is that the size of the array has to be as big as the number of processors. An alternative is to use a dynamic array wherein the array can be grown based on the number of simultaneous lock requestors. Give one advantage and one disadvantage of this idea compared to the MCS lock algorithm that uses a linked list to implement the lock as opposed to an array.

Ans:

- (+1.5) Advantage: arrays have better cache locality compared to linked lists
- (+1.5) Disadvantage: each element of the array has to be the size of the cache block to avoid false sharing leading to increased space complexity.

LRPC and Scheduling [12 points]

1. [8 points] A server provides the following procedure via LRPC

```
uint64_t sum_array(const uint64_t *arr, size_t len)
{
    uint64_t sum = 0;
    for (size_t i = 0; i < len; ++i) {
        sum += arr[i];
    }
    return sum;
}
```

The kernel will create an A-stack and map that into the client and server address spaces.

Assume the following:

- *(uint64_t is 8 bytes and size_t is 4 bytes)*
- There is programming language support for the server to access the A-stack directly.

a) [2 points] What is the minimum size (in bytes) of the A-stack?

Ans:

- 4 bytes (size_t len) + 8 bytes * len (uint64_t array) + 8 bytes (uint64_t result)

b) [2 points] If the server wishes to entertain simultaneous calls from multiple clients, how big should the A-stack be?

Ans:

- ((8 bytes * Max size of the dynamic array supported by the server) + 4 bytes for len + 8 bytes for result) * Number of simultaneous calls

c) [2 points] During the actual call what is the action on the client side so far as A-stack is concerned?

Ans:

- [+2] Client-stub copies the len variable, and contents of the array into the A-Stack.

d) [2 points] Prior to executing the server procedure, what is the action on the server side so far as A-stack is concerned?

Ans:

- [+2] Nothing. The arguments on the A-stack are directly available for the server procedure due the PL support.

2. [2 points] You are designing a scheduler for a shared memory multiprocessor. Mention one pro and one con for implementing the scheduler with a single shared queue of processes to be scheduled

Ans:

- Pro: will help with load balancing across all the processors

- Con: contention for the shared queue when many processors are looking for work

3. [2 points] You are designing a scheduler for a shared memory multiprocessor. Mention one pro and one con for implementing the scheduler with a per-processor queue of processes to be scheduled.

Ans:

- Pro: Possible to implement cache affinity based scheduling algorithms to populate the per-processor queues

- Con: More opportunity for load imbalance depending on the specifics of the cache affinity based scheduling policy used (e.g., fixed processor).

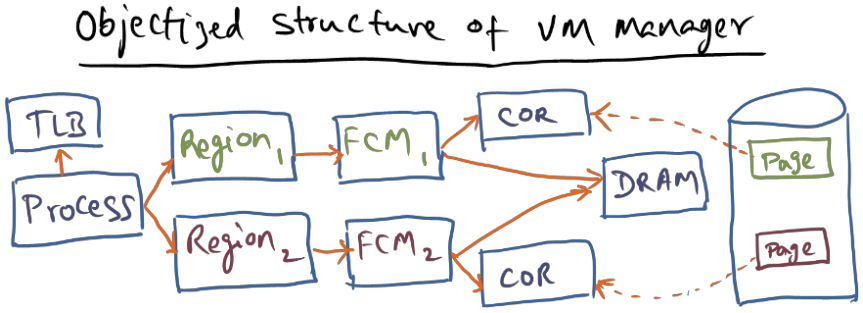
Tornado [6 points]

1. [2 points] In simple terms, give your understanding of the concept of a "clustered object" in the Tornado OS.

Ans:

Same object reference (i.e., logical) may de-reference to different object (i.e., physical).

2. a) [2 points] Using virtual memory management subsystem shown in the figure, elaborate how the clustered object abstraction of Tornado may be used in the implementation of the DRAM object for a multithreaded application running on a NUMA machine.



Ans:

Multiple instances of the DRAM object, one per NUMA piece of the memory; the clustered DRAM object will de-reference to the local NUMA piece of the requesting processor.

b) [2 points] How might Tornado decide to implement "Region" objects shown in the figure?

Ans:

Based on the page access patterns and page fault rate of the threads, Tornado would divvy up the address space into disjoint

regions and replicate each region object to increase concurrency of simultaneous page fault handling to different pages contained in any given region.

HIDDEN - Potpourri [21 points]

Full Virtualization [2 points]

1. [2 points] (True/False with justification) Every virtual memory access by a process in a fully virtualized environment goes through the guest OS to map the virtual page number (VPN) to physical page number (PPN); and then through the hypervisor to map the PPN to machine page number (MPN).

Ans:

- False
- Hypervisor directly keeps the mapping from VPN to MPN in the hardware page table and/or the TLB for the translation to happen at hardware speed.

Shared Memory [3 points]

[3 points] Answer True/False and provide justification for your response: Cache coherence is required to support a sequentially consistent memory model.

Ans:

[+1] False

[+1] Sequential consistency is a contract between the application programmer and the system.

[+1] The mechanism used to implement the memory consistency model could rely on coherent caches, but this is not required. Sequential consistency can also be implemented in software without hardware support.

Locks/Barriers [11 points]

1. [4 points] Consider the following lock algorithm using T&S:

```
while ((L == locked) or (T&S(L) == locked))
{
  while (L == locked); // spin
  delay (d[Pi]); // different delays for different processors
}
// success if we are here
```

a) [2 points] (Answer True/False with justification) (No credit without justification)

This algorithm does not rely on hardware cache coherence.

False.
Processors spin on cached value (While (L==locked)). The only way for them to come out of the spin loop is if the cached value changes which necessitates hardware cache coherence.

+0, Only False
+1, False with incorrect justification
+2, False with correct justification

b) [2 points] (Answer True/False with justification) (No credit without justification)
The algorithm performs especially well under high lock contention.

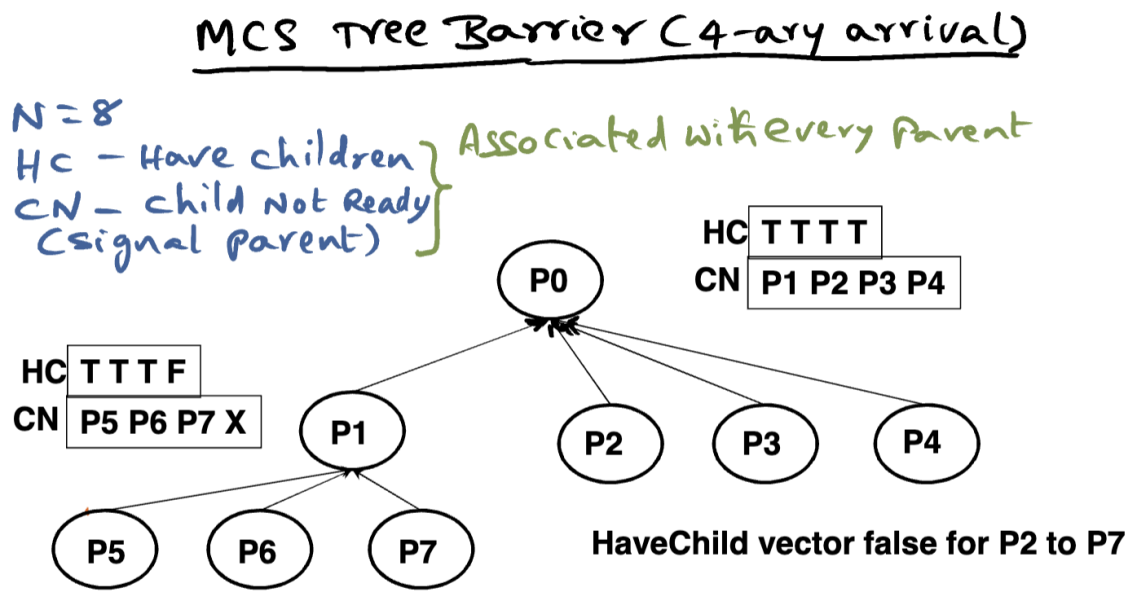
True.
Upon lock release, different processors wait for different amount of delay times thus reducing the contention on the interconnect.

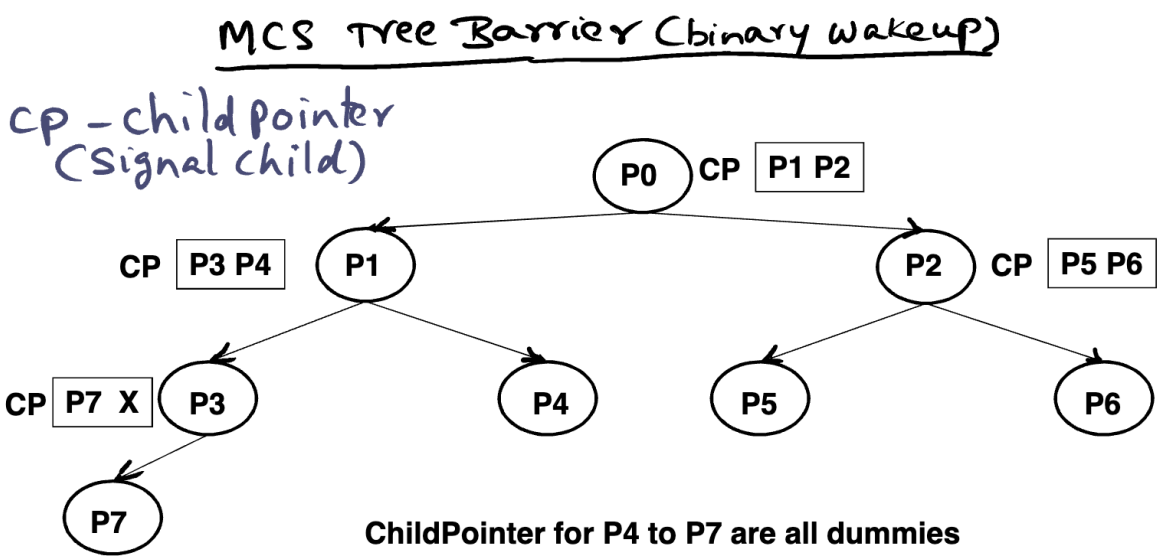
+0, Only True
+1, True with incorrect justification
+2, True with correct justification

2. [7 points] Answer the following questions with respect to the MCS barrier algorithm:

a) [2 points] (Answer True/False with justification) (No credit without justification)

For the correct functioning of the algorithm, it requires an atomic read-modify-write instruction to be supported in the architecture.





Ans: False.
By design, the data structures used in MCS for spinning ensure that there is exactly one writer and one reader.
Therefore, atomic read and atomic write instructions are sufficient to implement the algorithm.

b) [3 points] (Answer True/False with justification) (No credit without justification)

The algorithm will not work on an NCC-NUMA machine.

Ans: False. (+1)
(+2) The spin variable is statically assigned at the reading processor that is spinning on the variable in both the arrival and

wakeup trees. The remote write from the writing processor will be observed by the spinning processor since there is cache coherence internally within each node of the NCC-NUMA machine.

c) [2 points] (Answer True/False with justification) (No credit without justification)

The MCS algorithm works equally well on a shared memory multiprocessor as well as a cluster of processors with no physically shared memory.

Ans: False.
MCS algorithm assumes shared memory between the parent(s) and children, so it is not intended for a cluster where there is no physically shared memory.

Cellular Disco [5 points]

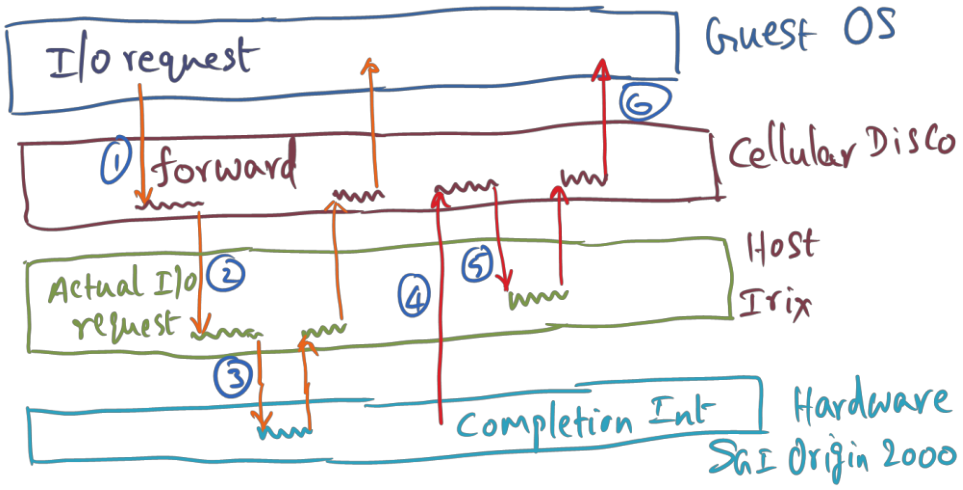


Figure shows I/O is virtualized in Cellular-Disco.

a) [3 points] What does cellular disco (CD) do in step 1 and why?

Ans:

(+1) I/O request from Guest OS intercepted by Cellular-Disco;

(+1) Rewrites the interrupt vector in Host's IVT so that the completion interrupt from the device (step 4) will come to Cellular-Disco.

(+1) It can then record on whose behalf (which Guest OS) the I/O was initiated for book-keeping and passing the I/O completion up to the right Guest OS (step 6).

b) [2 points] What does CD do in step 5 and why?

Ans:

+1) CD reactivates dormant Host OS; makes it appear as though the device completion interrupt just happened;

(+1) Allows clean up action of the device controller hardware by Host OS which cannot be done CD.